

Chương 5.
Lập trình C trên Linux

Nội dung

- Công cụ cần thiết
- Biên dịch chương trình đơn giản
- Tập tin tiêu đề (header file)
- Tập tin thư viện hàm
- Tiện ích make

Công cụ cần thiết

- Trình soạn thảo văn bản:
vi, gedit, emacs, geany, ...
- Trình biên dịch:
gcc/GNU, cc/Sun, bcc/Borland, g++/GNU, CC/Sun
- Thư viện chuẩn của ngôn ngữ C:
glibc

Biên dịch chương trình đơn giản

- Biên soạn chương trình sau và lưu vào tập tin có tên là hello.c

```
/*hello.c*/  
#include <stdio.h>  
main()  
{  
    printf("Hello, world!\n");  
    return 0;  
}
```

Biên dịch chương trình đơn giản (tt)

- Mở một terminal và dịch chương trình hello.c trên bằng trình biên dịch gcc của GNU
 - gcc hello.c
 - Tạo ra tập tin thực thi a.out
 - gcc -o hello hello.c
 - Tạo ra tập tin thực thi hello
 - gcc -c hello.c
 - Tạo ra tập tin mã đối tượng hello.o
 - Thực thi
 - ./a.out
 - ./hello

Biên dịch chương trình đơn giản (tt)

- Một số tùy chọn khác của gcc:
 - Wall: hiển thị toàn bộ các warning
 - ansi: Sử dụng C chuẩn ANSI
 - o: Đặt tên cho tập tin kết quả biên dịch
 - c: Tạo các tập tin đối tượng, không liên kết
 - lm: Liên kết với thư viện toán, nếu trong chương trình có `#include math.h`

Biên dịch chương trình đơn giản (tt)

- Một vài ví dụ về tùy chọn của gcc:
 - gcc -o hello hello.c
 - Tạo ra tập tin thực thi hello
 - gcc -c hello.c bonjour.c chao.c
 - Tạo ra các tập tin hello.o bonjour.o chao.o
 - gcc hello.o bonjour.o chao.o -o helloworld
 - Liên kết 3 tập tin mã đối tượng để tạo thành một tập tin thực thi helloworld.
 - Tập tin mã đối tượng giúp chỉnh sửa một tập tin không cần biên dịch lại các tập tin khác.

Tập tin tiêu đề (header file)

- Chứa các định nghĩa hằng, các khai báo về các hàm hệ thống hoặc hàm thư viện mà một chương trình C có thể gọi sử dụng.
- Lưu trữ mặc nhiên ở thư mục chuẩn /usr/include và các thư mục con của thư mục này
- Sử dụng tùy chọn -I khi biên dịch để tham khảo đến các tập tin tiêu đề ở một thư mục bất kỳ
 - Ví dụ: gcc -I/usr/openwin/include myprog.c

Tập tin thư viện hàm

- Chứa các hàm đã được biên dịch trước để có thể được sử dụng lại bởi các chương trình C khác mà không cần phải viết lại
- Các tập tin thư viện hàm chuẩn của hệ thống Linux được lưu trong thư mục `/lib` hoặc `/usr/lib`, ...
- Quy tắc đặt tên:
 - Thư viện tĩnh (static library): `libIndicat.a`
 - Thư viện chia sẻ (shared library): `libIndicat.so`
 - Ví dụ:
 - Thư viện hàm C: `libc.a`
 - Thư viện về toán: `libm.a`

Tập tin thư viện hàm (tt)

- Cách sử dụng các tập tin thư viện hàm:
 - Mô tả đường dẫn đến tập tin thư viện hàm
`gcc -o myprog myprog.c /usr/lib/libm.a`
 - Dùng tùy chọn `-l` và indicat của thư viện hàm
`gcc -o myprog myprog.c -lm`
 - Tìm trong thư mục thư viện hàm chuẩn hệ thống;
 - Sử dụng thư viện chia sẻ `libm.so` trước nếu tồn tại, nếu không sẽ dùng thư viện tĩnh `libm.a`
 - Dùng tùy chọn `-L` để bổ sung thư mục chứa thư viện hàm:
`gcc -o myprog -L/usr/openwin/lib myprog.c -lX11`

Tập tin thư viện hàm (tt)

- Cách thức xây dựng thư viện hàm: Giả sử **tạo một thư viện** gồm 2 hàm là hello và bonjour.
 - (1) - Tạo tập tin header **mylib.h** khai báo 2 hàm này.
//File name: mylib.h
void hello(char * name);
void bonjour(char *name);

Tập tin thư viện hàm (tt)

- Cách thức xây dựng thư viện hàm: Giả sử tạo một thư viện gồm 2 hàm là hello và bonjour.

(2) - Tạo các tập tin cài đặt cho 2 hàm được khai báo trong mylib.h

```
// File name: hello.c
#include <stdio.h>
void hello(char * name)
{
    printf("Hello %s\n",name);
}
// File name: bonjour.c
#include <stdio.h>
void bonjour(char *name)
{
    printf("Bonjour %s\n",name);
}
```

Tập tin thư viện hàm (tt)

- Cách thức xây dựng thư viện hàm: Giả sử tạo một thư viện gồm 2 hàm là hello và bonjour.

(3) - Các chương trình khác include mylib.h để có thể sử dụng các hàm hello() và bonjour.

```
// File name: helloworld.c
#include "mylib.h"
int main()
{
    hello("Hung");
    bonjour("Hung");
    return 0;
}
```

Tập tin thư viện hàm (tt)

- Cách biên dịch chương trình helloworld.c khi chưa tạo tập tin thư viện hàm:

```
gcc -c *.c
```

```
ls *.o
```

```
bonjour.o hello.o helloworld.o
```

Tập tin thư viện hàm (tt)

- Tạo tập tin thư viện hàm chứa mã đối tượng của các hàm hello() và bonjour()

ar crv libmylib.a hello.o bonjour.o

- Cách biên dịch chương trình helloworld.c sử dụng tập tin thư viện hàm libmylib.a:

gcc -o helloworld helloworld.o libmylib.a

Hoặc gcc -o helloworld helloworld.o -L. -lmylib

- Tiện ích nm: xem các hàm sử dụng trong một chương trình, thư viện.

nm helloworld

nm libmylib.

Tiện ích make

- Giới thiệu: make là một tiện ích lập trình giúp người lập trình.
 - Không phải đánh lại các câu lệnh biên dịch nhiều lần;
 - Tránh sai sót khi nhập các tùy chọn biên dịch từ bàn phím;
 - Tiết kiệm thời gian biên dịch chương trình vì không biên dịch lại các tập tin nguồn không có sửa đổi;
 - Dễ dàng phân phối phần mềm dưới dạng mã nguồn để người cài đặt biên dịch lại khi cài đặt hệ thống.

Tiện ích make (tt)

- Tập tin mô tả: make sử dụng tập tin mô tả có tên mặc định là makefile hoặc Makefile để chỉ dẫn make cách thức biên dịch/biên dịch lại một cách tự động một chương trình. Một tập tin mô tả bao gồm 3 thành phần sau.
 - Các mục tiêu (targets): thường là các tập tin thực thi hoặc các tập tin mã đối tượng cần tạo ra.
 - Những sự phụ thuộc (dependencies) để chỉ ra sự phụ thuộc của một mục tiêu vào các tập tin khác.
 - Các luật (rules) để chỉ ra cách thức tạo ra các mục tiêu.

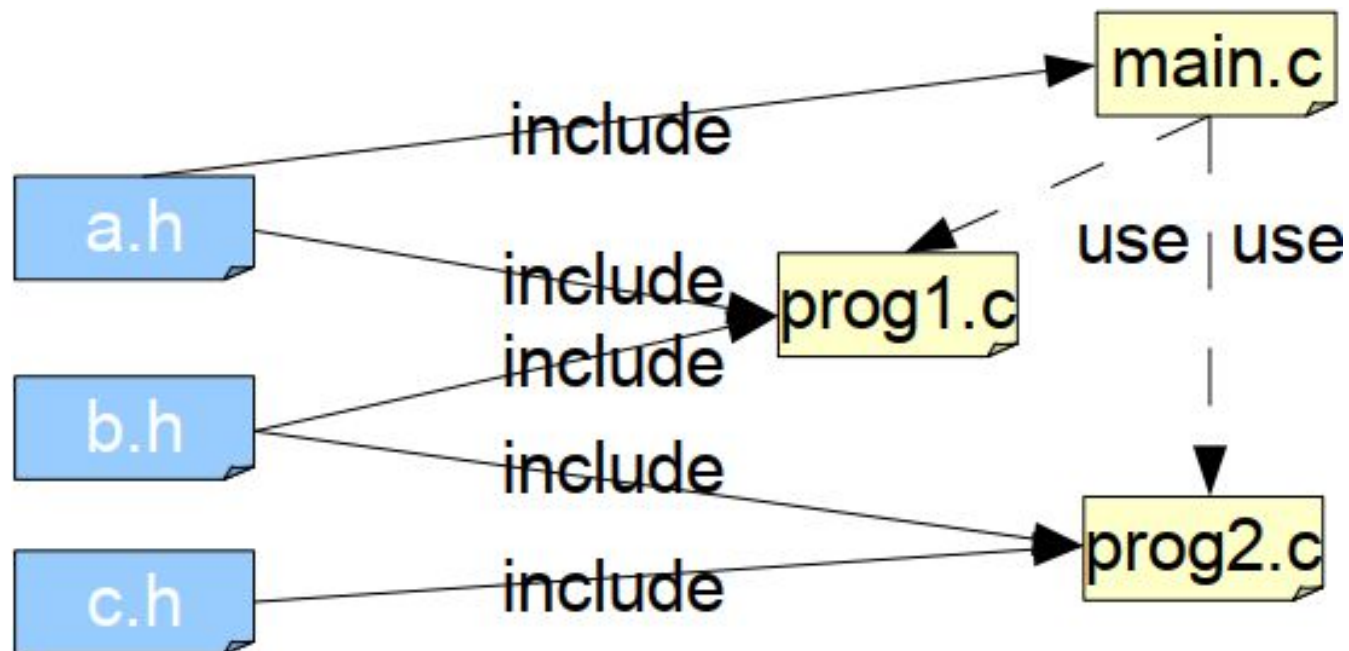
Tiện ích make (tt)

- Cách thức hoạt động:
 - make bắt đầu từ một mục tiêu được yêu cầu trong tập tin mô tả Makefile.
 - make kiểm tra xem mục tiêu hiện tại có phụ thuộc vào các mục tiêu khác không? Nếu có đi xuống một các đệ qui các mục tiêu con.
 - make dịch các tập tin nguồn thành các tập tin đối tượng, sau đó liên kết chúng lại thành tập tin thực thi.
 - make chỉ dịch lại tập tin nguồn thành tập tin đối tượng khi tập tin nguồn này bị sửa đổi.

Tiện ích make (tt)

- Xây dựng tập tin mô tả:

- Cho một dự án phần mềm phát triển bằng C có các tập tin phụ thuộc nhau như hình vẽ dưới đây.



Tiện ích make (tt)

- **Xây dựng tập tin mô tả:**
 - Tập tin Makefile để biên dịch dự án phần mềm này được viết như sau.

```
myapp: main.o prog1.o prog2.o
```

```
    gcc -o myapp main.o prog1.o prog2.o
```

```
main.o: main.c a.h
```

```
    gcc -c main.c
```

```
prog1.o: prog1.c a.h b.h
```

```
    gcc -c prog1.c
```

```
prog2.o: prog2.c b.h c.h
```

```
    gcc -c prog2.c
```

Tiện ích make (tt)

- **Xây dựng tập tin mô tả:**

Trong tập tin trên,

Các mục tiêu là: myapp, main.o, prog1.o, prog2.o. Đây là những cụm từ bắt đầu dòng và có ký tự hai chấm kết thúc.

Một mục tiêu sẽ phụ thuộc vào các tập tin hoặc các mục tiêu phụ khác, được liệt kê phía sau dấu 2 chấm của một mục tiêu. Ví dụ. mục tiêu myapp phụ thuộc vào các mục tiêu con main.o, prog1.o và prog2.o

Các luật là các dòng phía dưới mục tiêu và thụt vào đầu dòng 1 tab, ví dụ gcc -o myapp main.o prog1.o prog2.o

Tiện ích make (tt)

- Cú pháp sử dụng lệnh make:
 - make
 - Sử dụng tập tin makefile hoặc Makefile trong thư mục hiện hành như tập tin mô tả.
 - Tạo mục tiêu đầu tiên trong tập tin mô tả.
 - make -f MyMakeFile
 - Sử dụng tập tin MyMakeFile như tập tin mô tả.
 - make target-name
 - Tạo mục tiêu target-name trong tập tin mô tả.
 - Mục tiêu all thường được định nghĩa để bao gồm tất cả các mục tiêu.

Tiện ích make (tt)

- Sử dụng macro trong tập tin mô tả
 - Macro cho phép viết makefile một cách tổng quát và mềm dẻo hơn, tương tự như việc sử dụng biến và hằng trong lập trình.
 - Macro cho phép có nhiều tùy chọn cho việc biên dịch chương trình: phiên bản debug, phiên bản phát hành cũng như thay đổi trình biên dịch tùy thuộc vào hệ thống.

Tiện ích make (tt)

- Sử dụng macro trong tập tin mô tả
 - Định nghĩa macro: `MACRONAME=Value`
 - Truy cập giá trị:
`$(MACRONAME)`, `${MACRONAME}`, `$MACRONAME`

Tiện ích make (tt)

- Ví dụ một tập tin Makefile có sử dụng macro:

```
all: myapp
# Which compiler
CC = gcc
# Where are include files kept
INCLUDE = .
# Options for development
CFLAGS = -g -Wall -ansi
# Options for release
# CFLAGS = -O -Wall -ansi
myapp: main.o prog1.o prog2.o
    $(CC) -o myapp main.o prog1.o prog2.o
main.o: main.c a.h
    $(CC) -I$(INCLUDE) $(CFLAGS) -c main.c
prog1.o: prog1.c a.h b.h
    $(CC) -I$(INCLUDE) $(CFLAGS) -c prog1.c
prog2.o: prog2.c b.h c.h
    $(CC) -I$(INCLUDE) $(CFLAGS) -c prog2.c
```

Tiện ích make (tt)

- Một số macro sẵn dùng:
 - \$?: Danh sách các tập tin phụ thuộc có sửa đổi gần đây hơn so với mục tiêu hiện hành
 - \$@: Tên của mục tiêu hiện hành
 - \$<: Tên của tập tin phụ thuộc hiện hành
 - \$*: Tên của tập tin phụ thuộc hiện hành không có phần mở rộng
 - -cmd: Bỏ qua lỗi khi thực thi cmd
 - @cmd: yêu cầu make không in cmd ra màn hình trước khi thực thi nó

Bài tập

Bài 1: Viết một thư viện hàm có tên là: libmymath.a chứa các hàm toán học sau:

- Hàm tính giai thừa cho một số nguyên
- Hàm tính tổng chẵn từ 1 đến N
- Hàm tính tổng lẻ từ 1 đến N
- Hàm tính lũy thừa x mũ n .

Bài 2: Khai báo tập tin tiêu đề libmymath.h cho thư viện trên

Bài tập (tt)

Bài 3: Viết chương trình test-mymath sử dụng thư viện libmymath.a để tính và in ra màn hình kết quả các phép toán sau.

- Giai thừa của 10
- Tổng chẵn từ 1 đến 100
- Tổng lẻ từ 1 đến 100
- 8 lũy thừa 4.

Bài 4: Viết makefile với các mục tiêu sau.

- all: để tạo ra tất cả các mục tiêu
- test: để tạo ra chương trình test-mymath
- mymath: để tạo ra thư viện mymath

Bài 5: Chạy thử chương trình test-mymath